EC Optimization: Code Structure

Ulysse Faure

May 26, 2026

Contents

1 Introduction **1**

2 Grid search **1**

3 Other optimization schemes **1**

4 Systematic testing code **2**

1 Introduction

For the moment, the bulk of the code for EC Optimization is contained in the folder `torbeam-matlab/tools` (though other files were changed[1]). There are three main features I've worked on: making a robust, optimized grid search; designing other optimization schemes; and writing code to test them systematically.

2 Grid search

There are two relevant files: `torbeam_optimize_ec_using_angle_grid.m` and `torbeam_optimize_ec_demo.m`. The first is a function file and contains the actual code for grid search. The `demo` is a script the user can use to run the function file on a given shot with specified parameters (grid resolution, torbeam flavour, ...).

Grid search works in the following way: for a search window $(\theta_{\min}, \theta_{\max}) \times (\varphi_{\min}, \varphi_{\max})$ defined by the user, evaluate Torbeam on a grid of that search window. For each evaluation, record (essentially) all outputs of torbeam, in particular the ρ of maximal deposition $\rho_{\max}$ and the current drive efficiency $\eta = \frac{\text{driven current}}{\text{input power}}$. One obtains a $\rho_{\max}$ grid and an η grid. Using the built-in matlab function `contourc`, estimate the angles (θ, φ) for which $\rho_{\max} = \rho_{\text{target}}$. On these contours, find the minimal[2] value of η (where η is extended linearly between grid points). Return the corresponding $(\theta, \varphi)_{\text{opt}}$.

Table 1 records the main parameters of the grid search function.

3 Other optimization schemes

Again, there are two relevant files: `torbeam_test_optimizer.m` and `torbeam_test_optimizer_demo.m`. The first is a function file and contains the different EC optimization algorithms. The `demo` is a script

[1] for instance, `torbeam-matlab/torbeam_input.m`

[2] Since η is negative, except in countercurrent, this actually maximizes current drive.

the user can use to run the function file on a given shot with specified parameters (optimization algorithm, maximum time, ...).

The syntax of the `torbeam_test_optimizer` function is the following:

`function [thetaphi_opt, fval, exitflag, output, trials] = torbeam_test_optimizer(tbm, theta_bounds, phi_bounds, varargin)`. The main inputs of the function are described in Table 2, while its outputs are described in Table 3.

4 Systematic testing code

There are two relevant scripts: `prepare_optimization_data.m` and `read_optimization_data.m`. The first takes a series of triplets (shot, time, launcher), a list of optimizers, and other parameters as input. The code produces a series of folders, one per input[3], containing:

- The complete results of a grid search on that input;
- The complete results of each optimizer run on that input.

`prepare_optimization_data.m` typically runs at least for a few minutes or tens of minutes for each input (depending on the chosen parameters). If we give it a few tens of inputs it will run all night. See Table 4.

`read_optimization_data.m` reads the files contained in the relevant folders and produces graphs showing how fast the optimization decreases and where the optimizers choose to evaluate torbeam. See Table 5.

Table 1: Main input parameters in `torbeam_optimize_ec_using_angle_grid`.

Parameter	Description
shot, time, launcher	these are used to load the data (plasma configuration)
test_theta, test_phi	Correspond to `linspace`s of the angle ranges. Torbeam is evaluated at each `test_theta(i), test_phi(j)`.
`rho_request`	ρ_{target}.
`use_flavour_A`	If this is true, torbeam is evaluated in its fast (real-time) flavour. This flavour does not actually compute $\rho_{\max}$ for current drive, so $\rho_{\max}$ for power absorption is used as a proxy instead.
`plot`	If true, show results on a color map.

[3](shot, time, launcher)

[4]Careful: the weighting is done (so far) **differently** than in the optimization run by grid search, and requires reference values not yet put in. For the moment, just set parameter to 1.

Table 2: Main inputs of `torbeam_test_optimizer`.

Parameter	Description
`tbm`	Torbeam object, containing equilibrium data, etc.
`thetabounds, phibounds`	Should be something like $[10, 45], [-120, -180]$ - the grid bounds.
`solver`	Which solver to use: should be one of `surrogate, finite_differences, simulannealbnd, ga, Bayesian, Exploit_previousGridSearch`. In practice `finite_differences` and `ga` do not work well (yet).
`eccd_vs_width_weight`	$\in [0, 1]$, if 1, only consider maximal eccd, if 0, try to minimize deposition width.[4]
`rho_target, rho_tolerance`	The quantity to optimize (usually current drive) is first multiplied by $\exp\left(-\frac{(\rho_{\max}-\rho_{\text{target}})^2}{2\rho^2_{\text{tolerance}}}\right)$, where $\rho_{\max}$ is computed by torbeam.
`Maxtime`	Total time granted to the solver for the optimization
`Timeout`	Number of seconds granted for a torbeam run before the run is aborted

Table 3: Outputs of `torbeam_test_optimizer`.

Output	Description
`thetaphi_opt, fval`	vector $[\theta_{\text{opt}}, \theta_{\text{opt}}]$ according to optimizer, and the corresponding value $\eta \times \exp\left(-\frac{(\rho_{\max}-\rho_{\text{target}})^2}{2\rho^2_{\text{tolerance}}}\right)$.
`exitflag`	Is not zero if the optimization failed for some reason.
`output`	A structure describing the output, containing among others the total runtime
`trials`	Contains all points where torbeam was evaluated, along with the recorded fvalue $\eta \times \exp(\dots)$.

Table 4: Main functions in `prepare_optimization_data`.

Function name	Inputs	Description
run_and_save _grid_torbeams	save_folder, shots, times, launchers, theta_bounds, phi_bounds, grid_resolutions	For each element in the list (shots(i), times(i), launchers(i), ...), run the angle search and save it to `save_folder/<outputdir>/gridresults.mat`, where `outputdir` contains the shot, time, launcher info.
run_optimization _routines;	test_folder, solvers	For each `griddata.mat` found within `test_folder` (recursive search), run each optimization scheme in `solvers` on that shot configuration, and save results in `optimresults_vX.mat`, for the least integer X.

Table 5: Main functions in `read_optimization_data`.

Function name	Inputs	Description
compare_multiple _optim_data	griddata_vec, optimdata_vec	Plots optimization results (best ECCD) against time for each solver in `optim_data`, with data coming from a single shot and many time frames.
compare_single _optim_data	griddata, optimdata	Plots performance of optimizers against number of torbeam evaluations for the optimization recorded in `optimdata` (single time frame). Plots position of torbeam evals in (θ, φ) plane as well.
save_film _of_gridsearches	griddata_vec, folder	Checks the `griddata` (coming from grid search) are from the same shot, time-order them, and save a film of the plots in `folder`. Shows how ECCD as a function of launcher angles evolves during the shot.